Interrupts, the NVIC & Finite State Machines

IRQ – ISR – NVIC – FSM

Professor: Nerio Andrés Montoya
namontoy@unal.edu.co

2026-S2

Universidad Nacional de Colombia

Agenda

What is an Interrupt?

The NVIC

Timers: Our First Concrete Example

EXTI: A Second Concrete Example

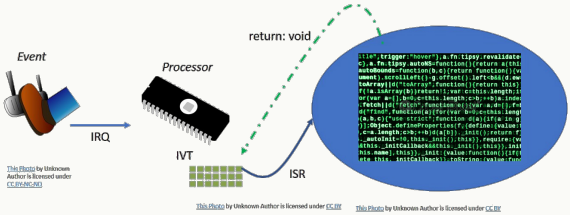
Finite State Machines

Closing

What is an Interrupt?

What is an Interrupt?

"An interrupt is an **asynchronous** event (also known as a 'request') that causes stopping the execution of the current code on a priority basis. If the request is accepted, the processor will suspend its current activities, save its state, and execute a function called an interrupt handler (or an interrupt service routine, **ISR**) to deal with the event."



You Already Know This



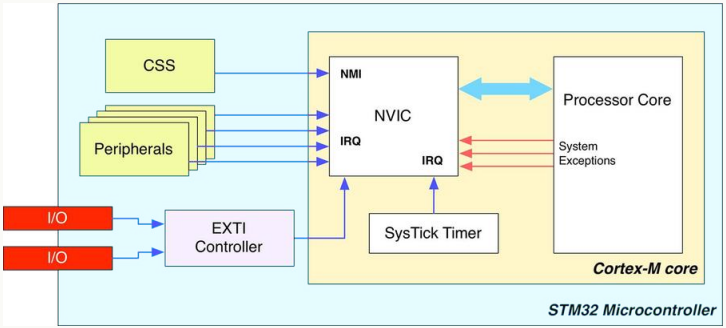
An alarm clock *interrupts* whatever you're doing — it doesn't wait for a convenient moment. That's exactly the behavior we're formalizing today.

Some Important Definitions

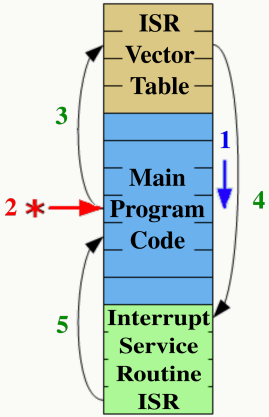
- **Interrupt**: the mechanism hardware uses to signal the processor that an **event** occurred
- **Interrupt Service Routine (ISR)**: the function the processor runs in response to that specific event
- For every interrupt, there is exactly one ISR
- **Interrupt Vector Table (IVT)**: a memory-mapped table linking each interrupt request to the memory address of its ISR
- Interrupts originate from hardware (and sometimes from firmware itself)
- On Cortex-Mx MCUs, interrupts are managed by a **Nested Vectored Interrupt Controller (NVIC)**

The NVIC

How Does It Work on ARM Cortex-M4?



General Flow of a Typical ISR



1. Ejecución normal del código principal.

2. Aparece un evento!
   Hay una interrupción (Push)

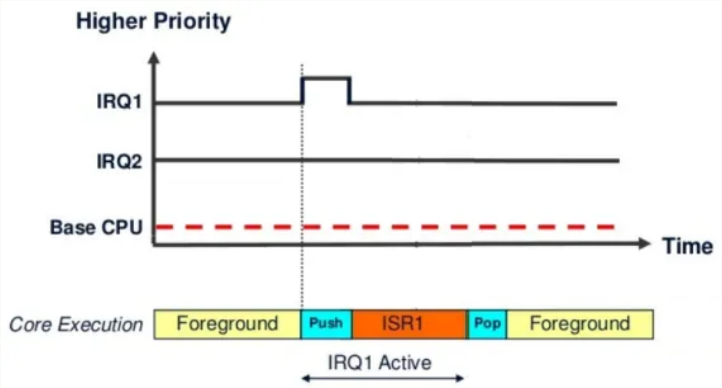
3. El programa "normal" se detiene y el sistema evalua la IVT.

4. Desde la IVT se salta a la ISR, donde la interrupción es atendida

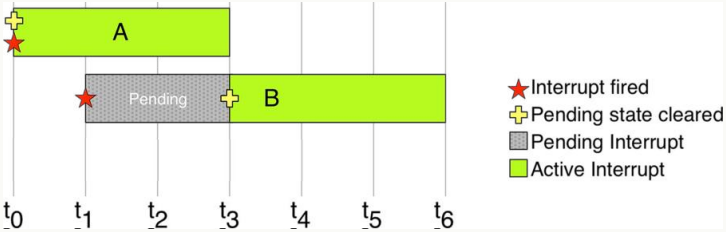
5. El sistema regresa al lugar donde estaba antes (Pop) y continua la ejecución normal

Saving Context: Push & Pop

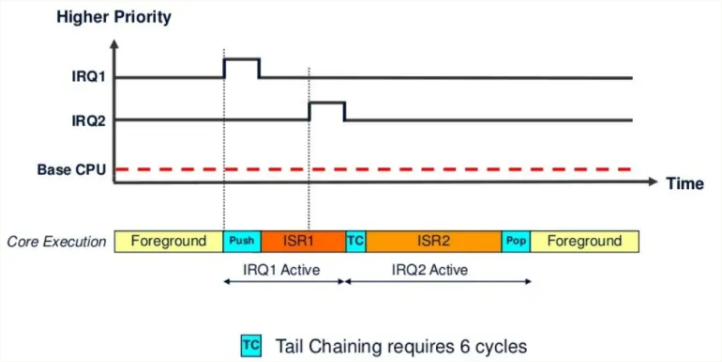
Save the current context so execution can resume exactly where it left off.



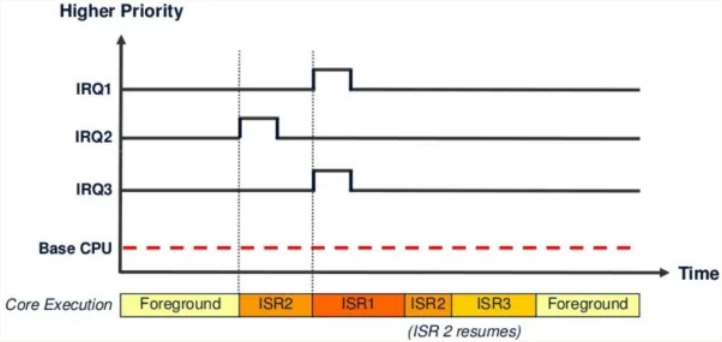
Interrupt Priority



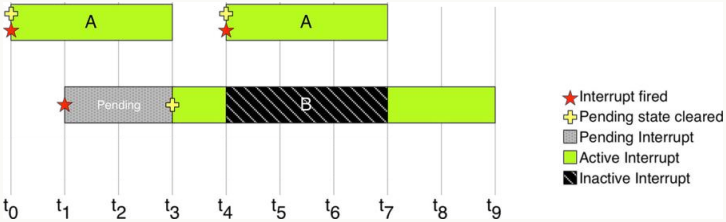
Priority — Tail Chaining



Interrupt Priority — Preemption



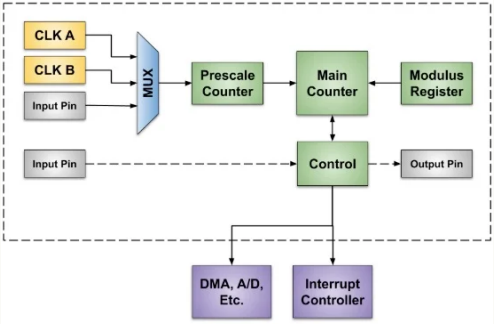
Interrupt Priority — Another Case



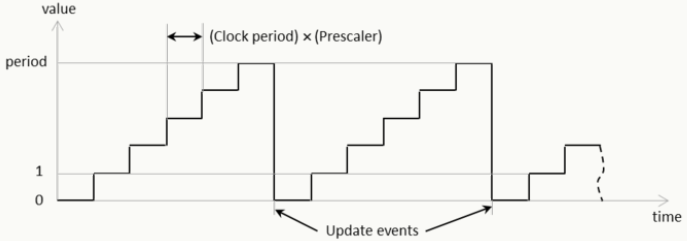
Timers: Our First Concrete Example

What is a Timer/Counter?

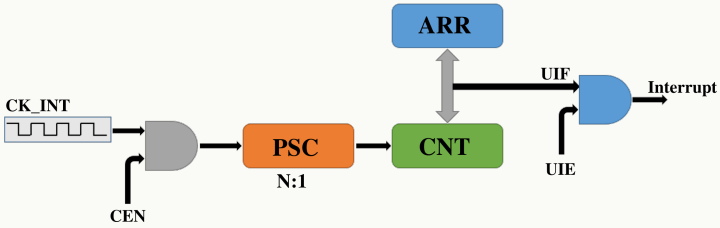
A **timer** is a specialized clock used to measure time intervals.
A **counter** stores (and sometimes displays) how many times an event occurred with respect to a clock signal — used to count events happening outside the microcontroller.



How Do They Work?



Hardware Configuration



EXTI: A Second Concrete Example

What is EXTI?

- **EXTI** (External Interrupt/Event Controller) lets a GPIO pin itself request an interrupt — no polling needed
- Same idea as our alarm: something happens (a button press, a sensor edge) and the processor reacts *immediately*, instead of checking that pin in a loop
- Each GPIO pin maps to an EXTI **line**; you choose to trigger on a rising edge, a falling edge, or both
- Once triggered, EXTI feeds the *same* NVIC we just covered — it's not a different mechanism, just a different **source**

EXTI + NVIC, Briefly

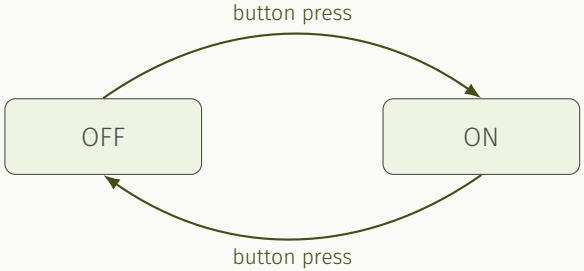
- `SYSCFG_EXTICRx` — picks *which* GPIO port drives a given EXTI line
- `EXTI_RTSR` / `EXTI_FTSR` — enable rising-edge / falling-edge triggering
- `EXTI_IMR` — unmasks the line so it can actually generate an interrupt
- Then: enable that line in the **NVIC**, same as any other interrupt source

This is the short version — we'll configure this in detail, register by register, in a later session.

Finite State Machines

What is a Finite State Machine?

A **Finite State Machine (FSM)** is a model where a system is always in exactly one **state**, and moves to another state only in response to an **event**.



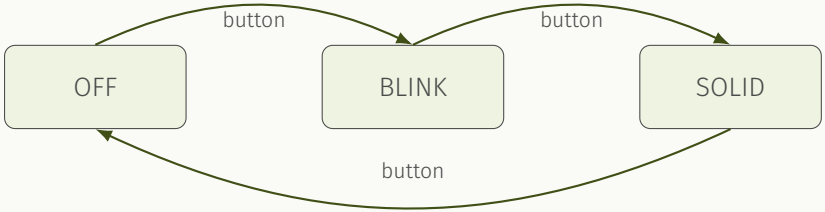
- **States**: the finite set of conditions the system can be in
- **Events**: what causes a transition — often, exactly the interrupts we've been discussing all class
- **Actions**: what happens on a transition, or while in a state

Why FSMs?

- Without one, interrupt-driven code tends to turn into a tangle of global flags and nested `if`s that only the original author can follow
- An FSM gives that same logic a **name** and a **picture** — the diagram *is* the documentation
- It also makes illegal transitions obvious: if there's no arrow for it, it shouldn't happen
- This is exactly how you should start structuring code that reacts to buttons, timers, and other interrupts

A Simple FSM Example

One button (via **EXTI**) cycles an LED through three modes:



- Same button, same EXTI line, same NVIC — only the *meaning* of the press changes, based on the current state

FSMs and Interrupts, Together

- Common, safe pattern: the **ISR** only sets a flag (“button was pressed”) — it does *not* run the FSM logic itself
- The **main loop** checks that flag and, if set, runs one FSM transition
- Why: ISRs should be short; long processing inside one blocks every other interrupt
- You now have every piece: an event source (EXTI), a dispatcher (NVIC), and a way to organize what happens next (FSM)

Closing

References

Most of the images in this presentation were taken from the following sites, and some were modified to help clarify the concepts presented. Some text was also taken from these sites and lightly adapted for the same purpose.

- www.codeinsideout.com
- medium.com
- www.linkedin.com
- en.wikipedia.org
- visualgdb.com
- www.allaboutcircuits.com